

Lab 4: Fitting a GEV to a tide gauge

2026-09-18

In this lab, you will fit a generalized extreme value (GEV) distribution to annual maximum water levels at a tide gauge by maximum likelihood, by L-moments, and by Bayesian inference, then compare the estimates.

The code below was written for Sewells Point, Virginia; this copy runs it on Seattle, Washington. Changing the `station` line is all it takes to select a gauge. A different gauge may need different detrending and prior choices.

Objectives

By the end of this lab, you will be able to:

1. Fit a GEV by maximum likelihood and by L-moments, and say why the two disagree.
2. Write a GEV log-likelihood from the density and optimize it yourself.
3. Read a prior predictive simulation and say whether the prior is defensible.
4. Diagnose a Markov chain that has not converged, and say what to do about it.

Setup

The cell below activates this lab's own package environment, installs what it lists, and loads the packages. Every lab starts with the same cell. It also creates the random number generator that the rest of the lab draws from, seeded so a fresh render gives the same figures every time.

Four of these packages are new this week.

- `CEVE543Utils` wraps GEV fitting and NOAA data loading for this course
- `Turing` writes Bayesian models and draws from their posteriors (documentation)
- `Optim` minimizes the likelihood you write by hand
- `Unitful` attaches physical units to numbers

```
using Pkg
Pkg.activate(@__DIR__)
Pkg.instantiate()

using CEVE543Utils
using CairoMakie, DataFrames, Distributions, LaTeXStrings
using Optim, Printf, Random, Statistics, Turing, Unitful
using FlexiChains: summarystats

rng = MersenneTwister(543)
```

```
set_theme!(theme_minimal()); fontsize=14)
colors = Makie.wong_colors()
```

Line 8

Turing stores its output in a `FlexiChain`, and `summarystats` is the function that turns one into the table of convergence diagnostics used at the end of this lab.

Line 10

Passed to every function below that draws at random, so the results are reproducible.

Lab activity

The data

`load_water_level` downloads every hourly reading a NOAA tide gauge has published and caches it beside this notebook. The Seattle gauge (station 9447130) sits on Elliott Bay in Puget Sound, and the record used here starts in 1928.

```
station = "9447130"
record = load_water_level(
    station;
    name="Seattle",
    cache=joinpath(@__DIR__, "$station-hourly.csv"),
)
```

Line 1

The line you will change later. Station ids are listed at NOAA's station browser; pick one with a long record.

Line 5

Where the download is kept. `@__DIR__` is this notebook's own folder, so the cache is found whether you render the notebook or run cells in the REPL. The first download of a century-long record takes a minute or two; every run after that reads the cache in seconds.

```
WaterLevelRecord: 863958 readings in m
station: Seattle (9447130), MSL datum
period: 1928-01-01T00:00:00 to 2026-08-31T23:00:00
levels: -3.547 to 2.57 m
```

The levels come back with units attached. `Unitful` attaches units to numbers, so a length cannot be added to a time and unit conversion is a function call rather than a hardcoded factor.

```
reading = record[1]
@printf("%s at %s\n", reading.level, reading.time)
@printf("in feet: %s\n", uconvert(u"ft", reading.level))
@printf("as a bare number: %.3f\n", ustrip(reading.level))
```

Line 3

`uconvert` changes the unit and the number together, so the physical quantity is unchanged.

Line 4

`ustrip` throws the unit away and hands back the number. Anything that does not understand units, which includes every fitting function in this lab, needs this.

```
-0.867 m at 1928-01-01T00:00:00
in feet: -2.8444881889763782 ft
as a bare number: -0.867
```

Annual maxima

```
annmax = AnnMaxRecord(record; detrend=:msl, units=u"ft")
```

Line 1

`:msl` subtracts each year's mean sea level, leaving the water level above that year's mean. `:linear` subtracts an ordinary least squares fit to the maxima instead, and `:none` leaves the trend in. Try each one.

```
AnnMaxRecord: 98 years in ft, detrended by :msl
station: Seattle (9447130), MSL datum
period: 1928 to 2025
levels: 6.347701801615551 to 8.497762399777082 ft
```

An observed water level is the sum of mean sea level, the astronomical tide, storm surge, and vertical land motion. Seattle's annual mean sea level has risen about 2.3 mm/yr over this record (a least-squares line through the annual means), about half the 4.6 mm/yr at Sewells Point. A stationary GEV requires that every year be drawn from the same distribution, so the trend must be removed first. Subtracting annual mean sea level removes the sea-level component but leaves the tide in the residual. A design study would remove the tide as well.

```
y = ustrip.(waterlevels(annmax))
raw = y .+ annmax.baseline .- mean(annmax.baseline[(end-4):end])

fig = Figure(; size=(700, 320))
ax = Axis(fig[1, 1]; xlabel="year", ylabel="Annual Maximum [ft]")
scatter!(ax, obsyears(annmax), raw; color=(:gray, 0.6), label="observed")
scatter!(ax, obsyears(annmax), y; color=colors[1], label="detrended")
lines!(ax, obsyears(annmax), annmax.baseline; color=colors[2], label="mean sea level")
axislegend(ax; position=:lb)
fig
```

Line 1

The fitting functions below all want bare numbers, so this is the vector every fit uses.

Line 2

Undoing the detrending to show what came in. `AnnMaxRecord` keeps the baseline it subtracted for exactly this reason.

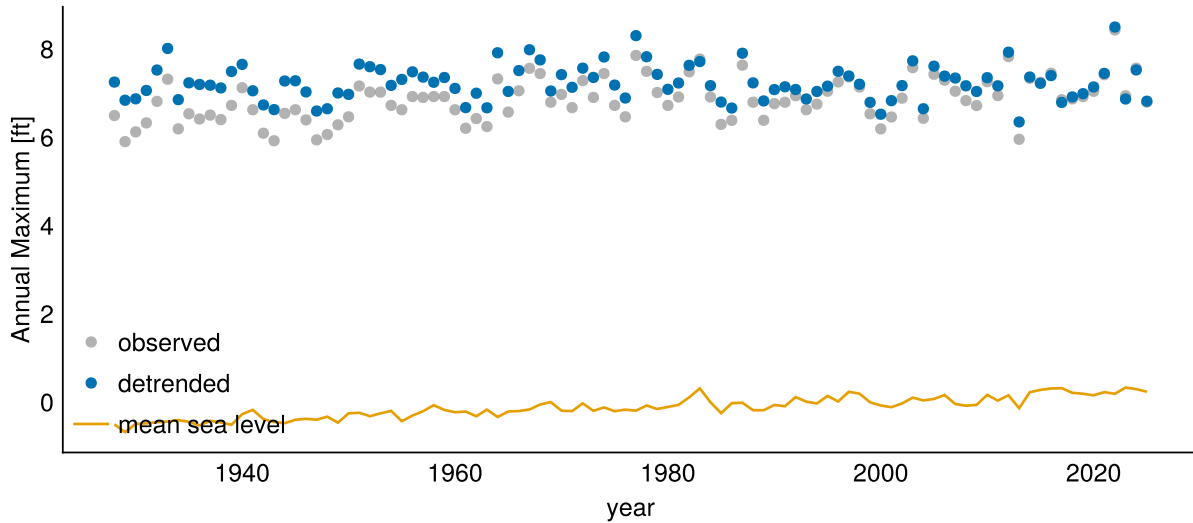


Figure 1: Annual maxima at Seattle before and after removing mean sea level. The baseline is what was subtracted.

Maximum likelihood and L-moments

`gevfit` fits by maximum likelihood and `gevfit_lmom` by L-moments. Both follow `Extremes.jl`.

```
fit_mle = only(getdistribution(gevfit(y)))
fit_lmom = gevfit_lmom(y)

@printf("MLE:      μ = %.3f, σ = %.3f, ξ = %.3f\n", fit_mle.μ, fit_mle.σ, fit_mle.ξ)
@printf("L-moments: μ = %.3f, σ = %.3f, ξ = %.3f\n", fit_lmom.μ, fit_lmom.σ, fit_lmom.ξ)
```

```
MLE:      μ = 7.069, σ = 0.356, ξ = -0.138
L-moments: μ = 7.068, σ = 0.357, ξ = -0.140
```

The GEV log-likelihood

You can also write the log-likelihood yourself. The log-density, with $v = 1 + \xi(y - \mu)/\sigma$, is:

$$\log f(y \mid \mu, \sigma, \xi) = -\log \sigma - \left(1 + \frac{1}{\xi}\right) \log v - v^{-1/\xi}.$$

Two cases need handling. When $v \leq 0$ the observation is outside the support and the density is zero, so the log-density is $-\infty$. When $\xi = 0$ the expression divides by zero, but the distribution is the Gumbel there and its log-density is $-\log \sigma - z - e^{-z}$ with $z = (y - \mu)/\sigma$.

```

function my_gev_logpdf(y,  $\mu$ ,  $\sigma$ ,  $\xi$ )
   $\sigma \leq 0$  && return -Inf
  z = (y -  $\mu$ ) /  $\sigma$ 
  abs( $\xi$ ) < 1e-7 && return -log( $\sigma$ ) - z - exp(-z)
  v = 1 +  $\xi$  * z
  v <= 0 && return -Inf
  return -log( $\sigma$ ) - (1 + 1 /  $\xi$ ) * log(v) - v^(-1 /  $\xi$ )
end

```

Line 2

A negative scale is not a distribution. Returning `-Inf` instead of raising an error lets the optimizer treat the point as impossible and continue.

Line 4

The Gumbel limit. Testing `abs( $\xi$ ) < 1e-7` rather than `$\xi == 0$` catches the values an optimizer proposes near zero.

Line 6

The support condition. As μ , σ and ξ change, the set of observations with finite density changes.

Now minimize the negative log-likelihood. Optimize over $\log \sigma$ so the scale stays positive.

```

function negative_loglik( $\theta$ , y)
   $\mu$ , log_ $\sigma$ ,  $\xi$  =  $\theta$ 
  return -sum(my_gev_logpdf(yi,  $\mu$ , exp(log_ $\sigma$ ),  $\xi$ ) for yi in y)
end

 $\theta_0$  = [mean(y), log(std(y)), 0.0]
result = optimize( $\theta \rightarrow$  negative_loglik( $\theta$ , y),  $\theta_0$ , NelderMead())
 $\hat{\mu}$ , log_ $\hat{\sigma}$ ,  $\hat{\xi}$  = Optim.minimizer(result)

@printf("converged: %s\n", Optim.converged(result))
@printf("by hand:    $\mu$  = %.3f,  $\sigma$  = %.3f,  $\xi$  = %.3f\n",  $\hat{\mu}$ , exp(log_ $\hat{\sigma}$ ),  $\hat{\xi}$ )
@printf("gevfit:     $\mu$  = %.3f,  $\sigma$  = %.3f,  $\xi$  = %.3f\n", fit_mle. $\mu$ , fit_mle. $\sigma$ , fit_mle. $\xi$ )

```

Line 3

Summing the log-densities and negating, since `optimize` minimizes.

Line 6

The starting guess: sample moments for location and scale, and zero shape (the Gumbel). A bad starting guess is the usual reason this fails. At Seattle a starting shape of 0.1 failed on the first step: a positive shape gives the GEV a lower bound, and the optimizer's first trial point raised that bound above the lowest observed year, so the likelihood was zero there. A zero shape has no bound.

Line 10

Check convergence before using the result.

```
converged: true
by hand:    $\mu = 7.069$ ,  $\sigma = 0.356$ ,  $\xi = -0.138$ 
gevfit:     $\mu = 7.069$ ,  $\sigma = 0.356$ ,  $\xi = -0.138$ 
```

! Important

If `converged` prints `false`, the parameters are not a maximum likelihood estimate and nothing downstream of them is valid. Try a different starting guess or a different algorithm.

Return levels from the three fits

Plot return level against return period, with the observations at Weibull plotting positions.

```
return_periods = exp.(range(log(1.1), log(500); length=60))
fit_hand = GeneralizedExtremeValue( $\mu$ , exp(log_ $\sigma$ ),  $\xi$ )

sorted_obs, _, observed_periods = plotting_positions(y)

fig = Figure(; size=(700, 400))
ax = Axis(fig[1, 1]; ylabel=L"\text{return level (ft)}")
for (d, colour, style, label) in (
    (fit_mle, colors[1], :solid, "gevfit (MLE)"),
    (fit_lmom, colors[2], :solid, "L-moments"),
    (fit_hand, colors[3], :dash, "by hand"),
)
    levels = [quantile(d, 1 - 1 / T) for T in return_periods]
    lines!(
        ax, return_periods, levels;
        color=colour, linewidth=2, linestyle=style, label=label
    )
end
scatter!(ax, observed_periods, sorted_obs; color=:black, markersize=6,
label="observed")
return_period_axis!(ax)
axislegend(ax; position=:lt)
fig
```

Line 1

Sixty return periods from just over 1 year to 500, log-spaced so the short ones do not crowd together.

Line 2

Your hand-fitted parameters, packaged as a distribution so one `quantile` call works on all three.

Line 4

Sorts the record and pairs each observation with the return period its rank implies.

Line 11

Dashed because your fit and `gevfit` maximize the same likelihood, so the two curves coincide. The dashed line following the solid one confirms your log-density is correct.

Line 13

The T -year return level is the quantile with exceedance probability $1/T$.

Line 20

Log axis with return periods as tick labels.

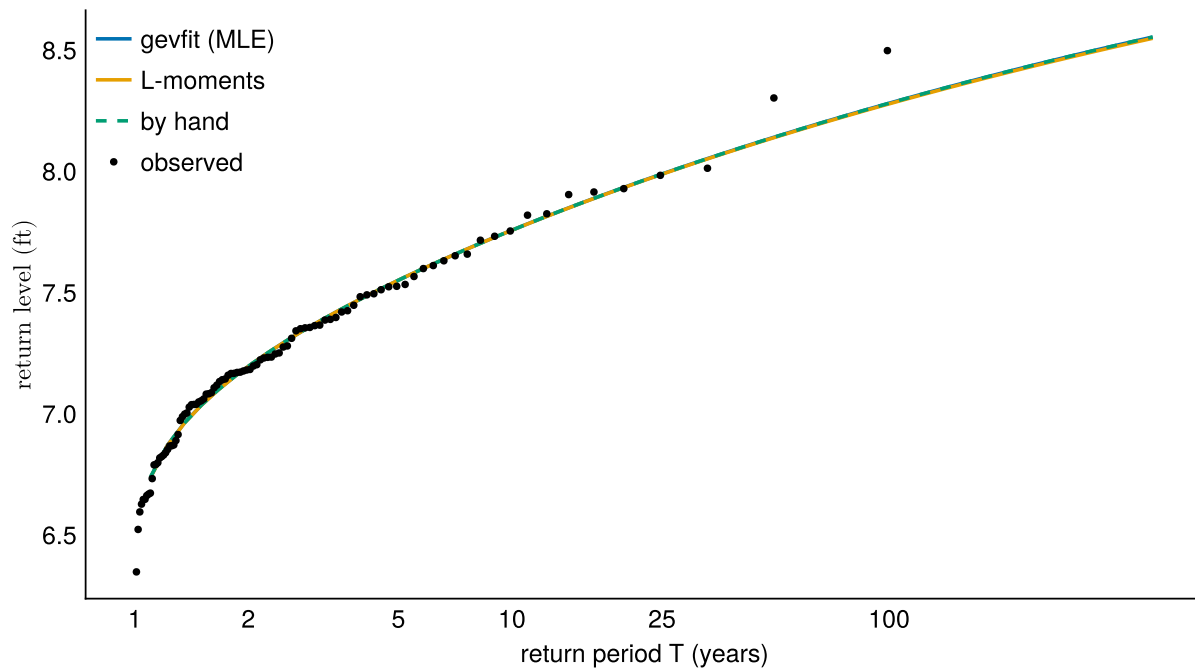


Figure 2: Return levels from the three fits, with the observed record at Weibull plotting positions.

Bootstrap confidence bands

Each of those curves is one estimate from 98 years of data. A different 98 years from the same process would give different parameters and a different curve. The bootstrap estimates that spread: resample the record with replacement, refit, and repeat.

Bootstrap the L-moment fit rather than the maximum likelihood fit. L-moments are closed form, so every resample returns an estimate. A maximum likelihood fit fails on some resamples, and the fits that survive are a biased subset.

```
n_boot = 1_000
boot_fits = [gevfit_lmom(y[rand(rng, 1:length(y), length(y))]) for _ in 1:n_boot]
boot_curves = [[quantile(d, 1 - 1 / T) for T in return_periods] for d in boot_fits]
```

Line 2

Resample `length(y)` values with replacement, refit, and keep the fitted distribution.

```

lower = [quantile([c[k] for c in boot_curves], 0.025) for k in
eachindex(return_periods)]
upper = [quantile([c[k] for c in boot_curves], 0.975) for k in
eachindex(return_periods)]

fig = Figure(; size=(700, 400))
ax = Axis(fig[1, 1]; ylabel=L"\text{return level (ft)}")
band!(ax, return_periods, lower, upper; color=(colors[2], 0.25), label="95% bootstrap")
lines!(
    ax, return_periods, [quantile(fit_lmom, 1 - 1 / T) for T in return_periods];
    color=colors[2], linewidth=2, label="L-moments"
)
scatter!(ax, observed_periods, sorted_obs; color=:black, markersize=6,
label="observed")
return_period_axis!(ax)
axislegend(ax; position=:lt)
fig

```

Line 6

`band!` fills between the two quantile curves.

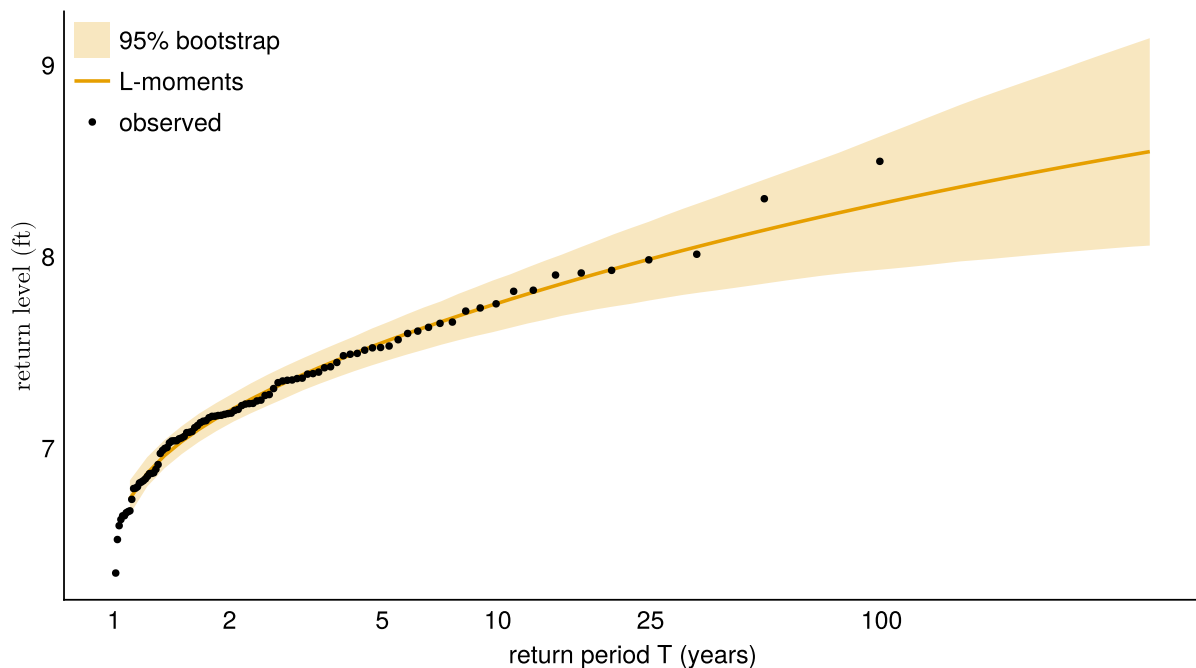


Figure 3: L-moment fit with a bootstrap 95% confidence band, against the observed record.

```

for T in (10, 50, 100, 500)
    levels = [quantile(d, 1 - 1 / T) for d in boot_fits]
    @printf("%3d-yr: %.2f [%.2f, %.2f] ft\n", T, quantile(fit_lmom, 1 - 1 / T),

```

```

    quantile(levels, 0.025), quantile(levels, 0.975))
end

```

```

10-yr: 7.76 [7.61, 7.89] ft
50-yr: 8.14 [7.86, 8.41] ft
100-yr: 8.28 [7.93, 8.63] ft
500-yr: 8.55 [8.06, 9.14] ft

```

The band widens with return period, because the far tail is determined by the few largest observations. Compare its width to the gap between the three fits in the previous figure: the disagreement between estimators is small next to the uncertainty within any one of them.

Bayesian estimation

The same model in Turing, with priors so wide they are meant to impose almost nothing. Priors this wide are often called uninformative.

```

@model function gev_vague(y)
    μ ~ Normal(0, 100)
    σ ~ truncated(Normal(0, 100); lower=0.01)
    ξ ~ Normal(0, 10)
    y .~ GeneralizedExtremeValue(μ, σ, ξ)
end

```

Line 3

Truncated so it stays positive.

Sampling the prior with `Prior()` ignores the data, so the draws show what the priors alone imply about water levels.

```

prior_df = DataFrame(sample(rng, gev_vague(y), Prior(), 1_000; progress=false))

```

Line 1

`Prior()` samples the prior rather than the posterior, which is what makes this a check on the prior alone.

```

function return_curves(df, periods; n_curves=200)
    curves = Vector{Vector{Float64}}()
    for i in 1:min(n_curves, nrow(df))
        df.σ[i] <= 0 && continue
        d = GeneralizedExtremeValue(df.μ[i], df.σ[i], df.ξ[i])
        curve = [quantile(d, 1 - 1 / T) for T in periods]
        all(isfinite, curve) && push!(curves, curve)
    end
    return curves
end

```

```

prior_curves = return_curves(prior_df, return_periods)
implied_500yr = [last(c) for c in prior_curves]
@printf("500-year level implied by the prior:\n")
@printf("      median  %.3g  ft,  largest  %.3g  ft\n",  median(implied_500yr),
maximum(implied_500yr))

```

Line 1

Turning each parameter draw into a return-level curve, which is the scale a coastal engineer can judge.

Line 7

A draw with ξ large enough gives infinite or overflowing quantiles, so those are dropped rather than plotted.

```

500-year level implied by the prior:
  median 173 ft, largest 5.75e+69 ft

```

```

fig = Figure(; size=(700, 340))
ax = Axis(fig[1, 1]; ylabel=L"\text{return level (ft)}", title="vague prior")
for curve in prior_curves
    lines!(ax, return_periods, curve; color=(colors[1], 0.06))
end
ylims!(ax, -20, 60)
hlines!(ax, [maximum(y)]; color=:black, linestyle=:dash, label="largest observed")
return_period_axis!(ax)
axislegend(ax; position=:lt)
fig

```

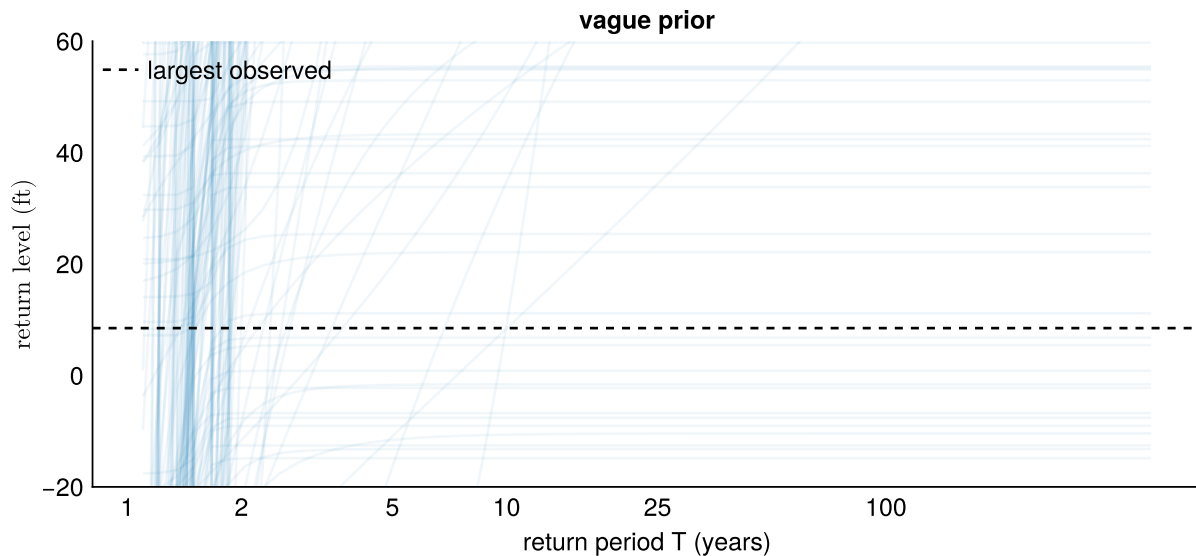


Figure 4: Return-level curves implied by the vague prior, before seeing any data. The vertical axis is clipped; many draws run far off the top.

These priors place substantial probability on 500-year water levels of several hundred feet and on negative water levels, neither of which any tide gauge has recorded. The prior is informative about return levels, and what it asserts is wrong.

Quantile priors

The three parameters are hard to reason about jointly, which is why independently reasonable priors on each one produced the curves above. Return levels are easier to judge, so specify the prior in terms of return levels instead. Choose a few return periods, state what you believe about the level at each, and let those statements constrain the parameters.

```
quantile_beliefs = [
  (period=2, belief=Normal(7.2, 0.5)),
  (period=100, belief=Normal(8.5, 1.0)),
  (period=500, belief=Normal(9.0, 1.5)),
]

@model function gev_quantile(y, beliefs)
  μ ~ Normal(7, 3)
  σ ~ truncated(Normal(0, 2); lower=0.01)
  ξ ~ truncated(Normal(0, 0.2); lower=-0.5, upper=0.5)
  d = GeneralizedExtremeValue(μ, σ, ξ)
  for b in beliefs
    Turing.@addlogprob! logpdf(b.belief, quantile(d, 1 - 1 / b.period))
  end
  for y_i in y
    Turing.@addlogprob! my_gev_logpdf(y_i, μ, σ, ξ)
  end
end
```

```
end
end
```

Line 1

Each entry is a belief about one return level: the 2-year level is 7.2 ft with standard deviation 0.5 ft, the 100-year level is 8.5 ft with standard deviation 1 ft, and the 500-year level is 9 ft with standard deviation 1.5 ft. **These numbers are for Seattle.** The Sewells Point values (5, 8 and 9.5 ft) put Seattle's typical year, about 7.2 ft above mean sea level, more than two standard deviations out.

Line 10

Bounding the shape rules out the distributions with no finite variance. This is a strong assumption; revisit it if the posterior for ξ concentrates near a bound.

Line 13

Each belief contributes its log-density evaluated at the return level the current parameters imply, so parameter sets implying return levels you called implausible receive lower posterior density.

```
quantile_prior_df = DataFrame(
    sample(rng, gev_quantile(Float64[], quantile_beliefs), NUTS(0.99), 1_000;
    progress=false)
)
quantile_curves = return_curves(quantile_prior_df, return_periods)

fig = Figure(; size=(700, 340))
ax = Axis(fig[1, 1]; ylabel=L"\text{return level (ft)}", title="quantile prior")
for curve in quantile_curves
    lines!(ax, return_periods, curve; color=(colors[1], 0.06))
end
hlines!(ax, [maximum(y)]; color=:black, linestyle=:dash, label="largest observed")
return_period_axis!(ax)
axislegend(ax; position=:lt)
fig
```

Line 2

Sampling the model with an empty data vector, so only the priors and the beliefs contribute. `Prior()` would not work here: it draws only from the `~` lines and skips every `@addlogprob!` term, so it ignores the beliefs entirely. On Seattle, `Prior()` gave a median 500-year level of about 15 ft against the 9 ft the beliefs state.

```
└ Info: Found initial step size
└   ϵ = 0.4
```

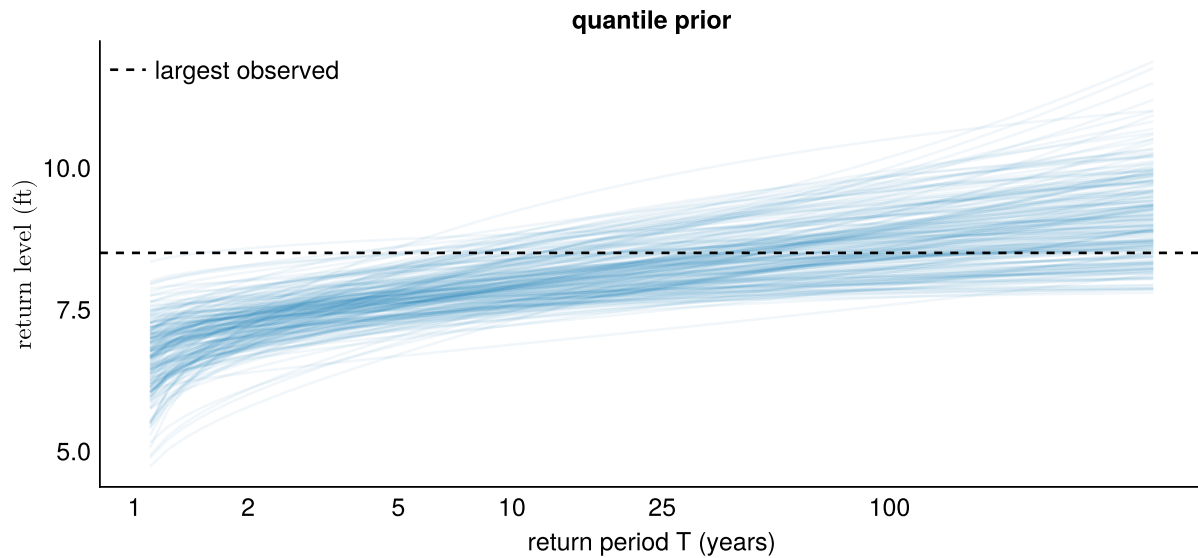


Figure 5: Return-level curves implied by the quantile prior. Compare the vertical scale to the previous figure.

```
for T in (2, 100, 500)
    levels = [
        quantile(GeneralizedExtremeValue(r.μ, r.σ, r.ξ), 1 - 1 / T)
        for r in eachrow(quantile_prior_df)
    ]
    @printf("%3d-yr level implied by the quantile prior: median %.2f [%.2f, %.2f] ft\n",
        T, median(levels), quantile(levels, 0.025), quantile(levels, 0.975))
end
```

```
2-yr level implied by the quantile prior: median 7.16 [6.32, 8.05] ft
100-yr level implied by the quantile prior: median 8.56 [7.46, 9.95] ft
500-yr level implied by the quantile prior: median 8.98 [7.61, 10.94] ft
```

The implied return levels are now physically plausible, and they still span a wide range. A prior predictive check that produces a narrow band means the prior determines the posterior.

Posterior fits and convergence diagnostics

```
chain_vague = sample(
    rng, gev_vague(y), NUTS(0.99), MCMCThreads(), 1_000, 4; progress=false
)
chain_quantile = sample(
    rng, gev_quantile(y, quantile_beliefs), NUTS(0.99), MCMCThreads(), 1_000, 4;
    progress=false,
)
```

Line 2

Four chains of 1,000 draws, run from different starting points. $\hat{R}$ compares chains against each other, so it needs more than one.

Line 5

Target acceptance rate, raised from the default of 0.65. The GEV support depends on the parameters, so the posterior density falls to zero along a boundary that moves with μ , σ and ξ . At the default step size NUTS crosses that boundary and reports a divergence. A higher target forces smaller steps, which costs a few seconds of runtime. Models whose support depends on their parameters usually need this; most models do not.

Check the chains before using them.

```
function check_chain(chain, label)
  stats = DataFrame(summarystats(chain))
  divergences = sum(chain[:numerical_error])
  @printf(
    "%-15s worst  $\hat{R}$  = %.3f, smallest ESS = %5.0f, divergences = %d\n",
    label, maximum(stats.rhat), minimum(stats.ess_bulk), divergences
  )
  return nothing
end

check_chain(chain_vague, "vague prior")
check_chain(chain_quantile, "quantile prior")
```

Line 1

Turing records a flag per draw; summing gives the count.

```
vague prior    worst  $\hat{R}$  = 3.717, smallest ESS =      8, divergences = 3830
quantile prior worst  $\hat{R}$  = 1.002, smallest ESS = 2180, divergences = 0
```

i Note

What the numbers mean. $\hat{R}$ compares the variation within each chain to the variation between chains; at convergence it approaches 1, and anything above 1.01 means the chains have not mixed. The effective sample size is how many independent draws the correlated ones are worth, so 4,000 draws with an ESS of 200 carry the information of 200. Divergences should be as close to zero as possible.

```
fig = Figure(; size=(700, 400))
ax = Axis(fig[1, 1]; ylabel=L"\text{return level (ft)}")
for (chain, colour, label) in
  ((chain_vague, colors[1], "vague prior"), (chain_quantile, colors[2], "quantile
  prior"))
```

```

curves = return_curves(DataFrame(chain), return_periods; n_curves=2_000)
lo = [quantile([c[k] for c in curves], 0.025) for k in eachindex(return_periods)]
hi = [quantile([c[k] for c in curves], 0.975) for k in eachindex(return_periods)]
med = [median(c[k] for c in curves) for k in eachindex(return_periods)]
band!(ax, return_periods, lo, hi; color=:black, 0.2)
lines!(ax, return_periods, med; color=:black, linewidth=2.5, label=label)
end
scatter!(ax, observed_periods, sorted_obs; color=:black, markersize=6,
label="observed")
return_period_axis!(ax)
axislegend(ax; position=:rb, framevisible=true)
fig

```

Line 6

The 2.5th and 97.5th percentiles across posterior draws at each return period, which bound the 95% credible interval.

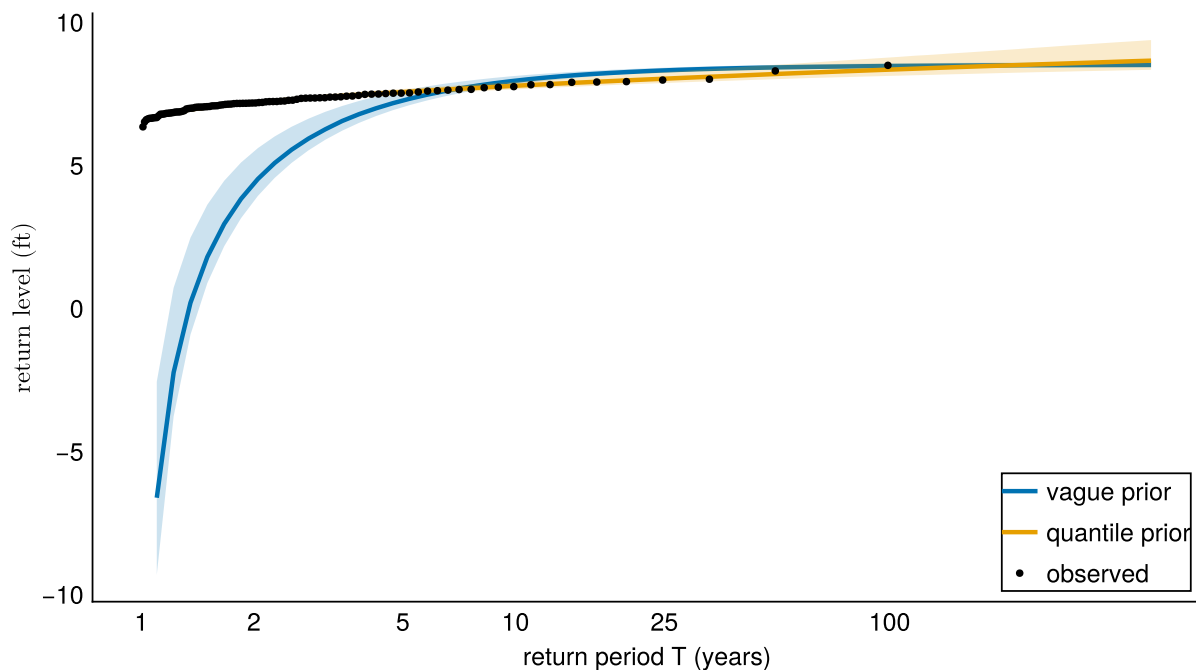


Figure 6: Posterior median return level and 95% credible interval under each prior. Shaded bands are the credible intervals; the observed record is plotted at Weibull plotting positions.

The two posteriors differ most at long return periods, where the data constrain the tail weakly.

```

function return_level_summary(chain, period)
    df = DataFrame(chain)
    levels = [
        quantile(GeneralizedExtremeValue(df.μ[i], df.σ[i], df.ξ[i]), 1 - 1 / period)
        for i in 1:nrow(df)
    ]
end

```

```

]
return median(levels), quantile(levels, 0.025), quantile(levels, 0.975)
end

for period in (50, 100, 500)
  vm, vl, vu = return_level_summary(chain_vague, period)
  qm, ql, qu = return_level_summary(chain_quantile, period)
  @printf(
    "%3d-yr   vague %5.2f [%5.2f, %6.2f]   quantile %5.2f [%5.2f, %6.2f]\n",
    period, vm, vl, vu, qm, ql, qu
  )
end

```

Line 1

Every posterior draw is a full parameter vector, so every draw gives a return level. The spread of those levels is the credible interval, and it accounts for uncertainty in all three parameters at once.

50-yr	vague	8.43 [8.38, 8.52]	quantile	8.19 [8.01, 8.52]
100-yr	vague	8.48 [8.44, 8.57]	quantile	8.35 [8.13, 8.77]
500-yr	vague	8.51 [8.49, 8.60]	quantile	8.66 [8.34, 9.37]

Exercises

You are expected to use an LLM for the code. Write your answers in the **Your answer** block under each exercise, replacing the italic placeholder. Grading is on the answers.

Change the gauge

Pick a different NOAA tide gauge from the station browser and change `station` in the data cell near the top. Choose one with at least 50 years of record. Then run the whole notebook from the top.

Report what broke and what you did about it. Work through the notebook in order and say, at each stage, whether the output is still trustworthy: did the hand-written optimizer converge, do the two fits still agree, is $\hat{R}$ still close to 1, are there divergences.

The quantile prior's numbers were chosen for Sewells Point. A gauge with a different tidal range, meaning the difference between high and low tide, needs different ones. Sewells Point has a tidal range of about 2.5 ft and Boston about 9.5 ft, so Boston's annual maxima exceed the levels this prior calls plausible. Look at your gauge's record before choosing new numbers, and say what you changed them to and why.

Your answer.

I switched to Seattle (station 9447130), which gives 98 annual maxima from 1928 to 2025. After removing each year's mean sea level they run from 6.35 to 8.50 ft, with a median of 7.18 ft. Mean sea level rose about 2.3 mm/yr over the record, half the Sewells Point rate. Going through the notebook in order:

- **Data and detrending:** loaded and detrended without problems. Trustworthy.

- **gevfit vs L-moments:** they agree closely (μ 7.069 vs 7.068 ft, σ 0.356 vs 0.357 ft, ξ -0.138 vs -0.140). Trustworthy.
- **Hand-written optimizer: broke.** It printed `converged: false` and returned its own starting guess ($\xi = 0.100$). Its negative log-likelihood (65.27) was the same as at the start, against 45.35 at the true optimum. With a starting ξ of +0.1 the GEV has a lower limit, $\mu - \sigma/\xi$. Nelder-Mead's first trial point moved μ up to 10.87 ft, which raised that limit to 6.93 ft, above Seattle's lowest year of 6.35 ft. The likelihood was zero there, so the optimizer stopped after 0 iterations. I changed the starting ξ to 0 (the Gumbel, which has no limit). It then converges and matches `gevfit` to three decimals.
- **Bootstrap:** ran without problems. Trustworthy.
- **Vague prior: broke, and I left it as is.** Worst $\hat{R} = 3.72$, smallest ESS = 8, and 3830 of 4000 draws diverged. The chains are stuck near $\xi \approx -1.07$, $\sigma \approx 6$ ft, $\mu \approx 3$ ft. Below $\xi = -1$ the GEV density goes to infinity at its upper limit, and the sampler chases that spike instead of fitting the data. Raising the target acceptance would not fix this, because the prior itself allows those values. Nothing from this chain is usable.
- **Quantile prior:** with the Sewells Point numbers it happened to converge ($\hat{R}$ 1.003, 1 divergence). But its beliefs put the 2-year level at 5 ± 1 ft, while Seattle's typical year is about 7.2 ft, more than two standard deviations away. I changed the beliefs to:
 - ▶ 2-year: Normal(7.2, 0.5), because the median annual maximum is 7.18 ft.
 - ▶ 100-year: Normal(8.5, 1.0), because the largest value in 98 years is 8.50 ft, which is roughly what a 100-year level means.
 - ▶ 500-year: Normal(9.0, 1.5), a little higher and wider because it is an extrapolation.

I also moved the prior on μ from Normal(5, 3) to Normal(7, 3). I made the standard deviations much smaller than at Sewells Point because Seattle's year-to-year spread is small ($\sigma \approx 0.36$ ft): its annual maximum is mostly a big tide. I kept them wider than that spread so the prior guides the fit without dictating it. One weakness: I read these beliefs off the same record I then fit, so the data count twice. The wide standard deviations are there to limit that. With the new prior, $\hat{R} = 1.002$ and there are 0 divergences.

- **Prior predictive check for the quantile prior: broke without any warning.** `Prior()` draws only from the `~` lines and skips the `@addlogprob!` beliefs, so the figure showed a median 500-year level of about 15 ft, not the 9 ft I asked for. I replaced it by sampling the model with no data. The prior now implies a 500-year level of 8.98 ft [7.61, 10.94].

The shape parameter

Compare the maximum likelihood, L-moment, and hand-optimized estimates on your gauge. Report all three.

They should agree closely on μ and σ and less closely on ξ . Explain why ξ is the least well determined, and say which of the three you would report to a client and why.

```
for (name, get) in (("mu", d -> d.mu), ("sigma", d -> d.sigma), ("xi", d -> d.xi))
  draws = get.(boot_fits)
  @printf("%s: estimate %6.3f, bootstrap sd %3f, 95%% [%6.3f, %6.3f], sd/|estimate|
= %2f\n",
    name, get(fit_lmom), std(draws), quantile(draws, 0.025), quantile(draws, 0.975),
```

```
std(draws) / abs(get(fit_lmom)))
end
```

Line 1

The bootstrap fits from earlier, read one parameter at a time, to show how well the record pins down each one.

```
μ: estimate 7.068, bootstrap sd 0.038, 95% [ 7.002, 7.148], sd/|estimate| = 0.01
σ: estimate 0.357, bootstrap sd 0.028, 95% [ 0.299, 0.408], sd/|estimate| = 0.08
ξ: estimate -0.140, bootstrap sd 0.075, 95% [-0.298, -0.016], sd/|estimate| = 0.53
```

Your answer.

Estimator	μ (ft)	σ (ft)	ξ
Maximum likelihood (<code>gevfit</code>)	7.069	0.356	-0.138
L-moments	7.068	0.357	-0.140
By hand (Nelder-Mead, starting at $\xi = 0$)	7.069	0.356	-0.138

The hand-written fit matches `gevfit` exactly because both maximize the same likelihood. At Seattle the three agree on ξ as well, only 0.002 apart. That agreement hides how poorly ξ is determined. In the bootstrap, ξ has a standard deviation of 0.075 and a 95% range of -0.30 to -0.02, about half its own size. μ varies by only about 1% of its value and σ by about 8%.

μ and σ describe the middle and spread of the annual maxima, and all 98 years contribute to them. ξ describes how the upper tail bends, and that depends on the few largest years. Only two years at Seattle are above 8.1 ft (8.50 and 8.30 ft). Whether a bootstrap sample happens to include them changes ξ a lot, and a different century of weather would have produced different extremes. Ninety-eight years hold very little information about the tail.

For a client I would report the L-moment fit with its bootstrap interval. Here it gives the same 100-year level as maximum likelihood (8.28 ft), so the choice does not change the number. But L-moments always return an estimate, so the bootstrap interval is reliable and does not silently drop failed fits. For records of about 100 years, L-moment (probability-weighted moment) estimates have been found to do as well as or better than maximum likelihood (Hosking, Wallis and Wood, 1985). The hand-optimized fit is a check that I wrote the likelihood correctly, not a separate estimate to report.

Sensitivity to the prior

Compare the 500-year return level and its credible interval under the two priors on your gauge.

The vague prior is sometimes described as letting the data speak for themselves. Using your prior predictive figures and your posterior intervals, say whether that description holds here.

Your answer.

	500-year level (ft)	95% interval (ft)	Width
Vague prior	8.51	[8.49, 8.60]	0.11 ft
Quantile prior	8.66	[8.34, 9.37]	1.03 ft
Bootstrap, for comparison	8.55	[8.06, 9.14]	1.08 ft

The description does not hold at Seattle. The vague prior is not neutral. Its prior predictive figure puts the median 500-year level at 173 ft. Of the 200 curves plotted, about a fifth have a negative 500-year level and about three quarters exceed 50 ft. It spreads its belief over values no tide gauge has ever recorded, and it allows shape values below -1 , where the likelihood has no maximum. The data cannot overcome that. The chains never converged ($\hat{R} = 3.72$, 3830 divergences), and they got stuck where $\xi \approx -1.07$. There the fitted distribution's upper limit sits at about 8.5 ft, the largest value in the record. So its very narrow 500-year interval says, in effect, "the worst water level possible is the worst we have seen." That comes from a stuck sampler, not from the data, and a city could be badly misled by it.

The quantile prior is where the data actually do the talking. Before seeing any data, it implied a 500-year level of 8.98 ft [7.61, 10.94], a range of 3.3 ft. After the data, the posterior was 8.66 ft [8.34, 9.37]: the median moved down, and the range shrank to about a third of the prior's. The posterior also sits close to the bootstrap estimate, which uses no prior at all. So the prior that states real beliefs about water levels was the one that let the data dominate. The one meant to be "uninformative" either overwhelmed the data or broke the sampler.

Reading a failed diagnostic

Suppose a colleague sends you this notebook, run on their own gauge, with $\hat{R} = 1.08$ on ξ and 47 divergences, and asks whether the 100-year level in the summary is usable.

Write your reply. Say what the two numbers mean, whether the estimate can be used, and what you would ask them to try.

Your answer.

Hi, thanks for sending this. Unfortunately I don't think the 100-year level is usable yet, not even as a rough number.

$\hat{R} = 1.08$ on ξ means your four chains disagree about the shape parameter. $\hat{R}$ compares the spread between chains with the spread within each chain. If the chains had all explored the same distribution it would be about 1.00, and the usual cutoff is 1.01. At 1.08 the chains are still exploring different ranges of ξ .

47 divergences means the sampler failed numerically 47 times and did not properly explore those regions. With the GEV this usually happens along the edge where the data fall outside the fitted distribution's range, and that edge moves as ξ changes. Divergences make the draws biased, not just noisy, so running more draws does not fix them.

Both problems are in ξ , and ξ is the parameter that controls the 100-year level most. So both the median and the interval could be wrong, and we can't tell by how much.

Could you try these steps?

1. Compare the posterior with `gevfit` and `gevfit_lmom` on the same data. If they disagree a lot, the chain is the problem.
2. Check which prior you used. If it's the vague one, look at its prior predictive figure. On my gauge (Seattle) the vague prior gave $\hat{R} = 3.7$ and 3830 divergences, and a quantile prior with beliefs for your own gauge fixed both.
3. Plot the chains for ξ to see whether one chain is stuck somewhere different from the others.
4. Check the data: units, missing or partial years, the detrending choice, and any suspicious outliers.
5. If it still struggles, run a longer warm-up, or tighten the bounds on ξ if the posterior is piling up against one.

Please rerun until $\hat{R}$ is below 1.01 and the divergences are at or near zero, and then report the 100-year level with its interval.

Reporting a design value

Report one 100-year return level for your gauge, with one interval, as you would for a report a city will act on.

State the number and say what it rests on: which estimator, which detrending choice, which prior, and what you would want before trusting it further.

```
for detrend in (:none, :linear, :msl)
    yd = ustrip.(waterlevels(AnnMaxRecord(record; detrend=detrend, units="ft")))
    d = gevfit_lmom(yd)
    @printf("detrend = %-8s  $\xi$  = %6.3f, 100-yr %.2f ft\n", detrend, d. $\xi$ , quantile(d, 1
- 1 / 100))
end

post = DataFrame(chain_quantile)
bound = [r. $\xi$  < 0 ? r. $\mu$  - r. $\sigma$  / r. $\xi$  : Inf for r in eachrow(post)]
@printf("posterior share with  $\xi$  < 0: %.2f; implied upper limit median %.2f ft, 2.5%
%.2f ft\n",
    mean(post. $\xi$  .< 0), median(bound), quantile(bound, 0.025))
```

Line 1

The same L-moment fit under each detrending choice, to see how much the design value depends on it.

Line 8

A negative shape gives the GEV a finite upper limit, $\mu - \sigma/\xi$, which no water level can exceed.

```
detrend = none       $\xi$  = -0.203, 100-yr 8.05 ft
detrend = linear     $\xi$  = -0.181, 100-yr 8.31 ft
detrend = msl        $\xi$  = -0.140, 100-yr 8.28 ft
posterior share with  $\xi$  < 0: 0.97; implied upper limit median 10.15 ft, 2.5% 8.82 ft
```

Your answer.

100-year water level at Seattle: 8.35 ft above that year's mean sea level, 95% credible interval 8.13 to 8.77 ft.

What this number rests on:

- **Estimator:** the Bayesian GEV posterior under the quantile prior. Its chains converged ($\hat{R} = 1.002$, 0 divergences, effective sample size about 2,200), and its interval carries the uncertainty in all three parameters at once. It agrees with the maximum likelihood and L-moment fits (8.28 ft) and with the bootstrap interval of 7.93 to 8.63 ft. It is a little wider on the high side, which is the safer side for design.
- **Detrending:** each year's mean sea level is subtracted. The number is therefore a height above mean sea level, not an elevation. To get a design elevation, the city has to add the mean sea level expected over the structure's life and convert to its own vertical reference. Over this record Seattle's mean sea level rose about 2.3 mm/yr (about 0.75 ft per century), and future rise is expected to be faster. The choice matters less than that addition: subtracting a straight-line trend instead gives 8.31 ft. Leaving the trend in gives 8.05 ft, which mixes old, lower sea levels into the fit and is biased low for today.
- **Prior:** my Seattle quantile beliefs (2-year 7.2 ± 0.5 ft, 100-year 8.5 ± 1 ft, 500-year 9 ± 1.5 ft). They were read off the same record, so the interval may be somewhat too narrow.
- **Model:** a single GEV that stays the same over time, fit to 98 annual maxima, with the tide left in. In 97% of the posterior draws ξ is negative, which gives the fitted distribution a hard upper limit (median 10.15 ft, and as low as 8.82 ft in the lowest 2.5% of draws). That limit is a feature of the curve, not a physical guarantee, and the city should not treat it as one.

Before trusting it further, I would want:

1. Sea-level rise projections added for the design year.
2. The tide and storm surge analyzed separately, so we know how much of an extreme comes from a very high tide and how much from a storm.
3. A comparison with NOAA's published extreme water levels for Seattle and with nearby gauges.
4. A model whose parameters can change over time, to test whether the extremes themselves are changing.
5. A check for missing or partial years in the record.
6. A check that the answer holds up under reasonable changes to the prior.

Turning it in

This lab is due on Canvas one week after the Friday session that starts it.

1. Check that every **Your answer** block is filled in.
2. Render the notebook to a PDF. In the terminal, from the lab folder, run:

```
quarto render index.qmd --to typst
```

This writes `index.pdf` beside the notebook. It runs the whole notebook from the top in a fresh process, so it also confirms your work does not depend on leftover REPL state.

3. Commit and push your work, with GitHub Desktop or by asking Claude to commit and push.
4. Upload `index.pdf` to the Lab 4 assignment on Canvas.
5. Paste the link to your repository in the same Canvas submission.

If the render fails and you cannot fix it, upload what you have and say where it broke. A lab that does not render is worth more to me than a lab you did not turn in.

Note what broke, what Claude got wrong, and what you had to fix by hand, and bring it to the next session.